ARTIFICIAL INTELLIGENCE (UNC504)
LAB FILE

Submitted By :

Siddharth Tiwari

102206080

3EC10/3F2E

Submitted To :

Ms. T. PUVIYARASI

THAPAR INSTITUTE OF ENGINEERING & TECHNOLOGY
(DEEMED TO BE UNIVERSITY)
PATIALA, PUNJAB, INDIA
DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

EVEN SEMESTER(JAN-JUL)

Experiment-1

BASIC OF PYTHON

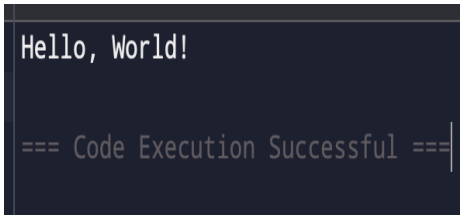
1. Printing and Variables

1.1 Write a program to print "Hello, World!"

Code:

```
print("Hello, World!")
```

Result:

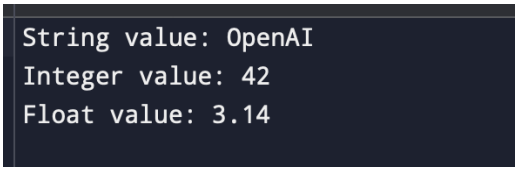


1.2 Declare variables of different data types (string, integer, float) and print their values

Code

```
# Variable declarations
my_string = "OpenAI"
my_integer = 42
my_float = 3.14
# Printing values
print("String value:", my_string)
print("Integer value:", my_integer)
print("Float value:", my_float)
```

Result:

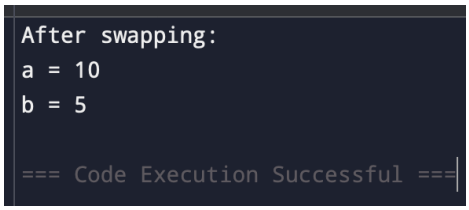


1.3 Swap the values of two variables without using a third variable

Code

```
# Initial values
a = 5
b = 10
# Swapping values
a, b = b, a
# Printing results
print("After swapping:")
print("a =", a)
print("b =", b)
```

Result:



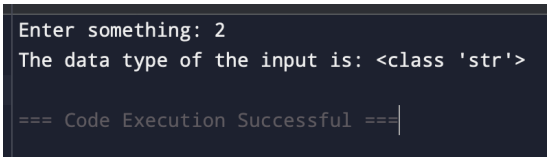
2. Data Types and Type Conversion

2.1 Take input from the user and print the data type of the entered value

Code:

```
user_input = input("Enter something: ")
print("The data type of the input is:", type(user_input))
```

Result:

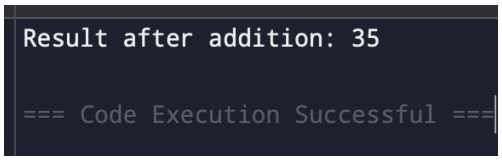


2.2 Convert a string to an integer and perform arithmetic operations

Code:

```
string_num = "25"
integer_num = int(string_num)
# Arithmetic operations
result = integer_num + 10
print("Result after addition:", result)
```

Result:



2.3 Explore implicit and explicit type conversions in Python

Code:

```
# Implicit conversion
a = 5
b = 2.5
c = a + b  # int + float -> float
print("Result of implicit conversion:", c, "| Type:", type(c))
# Explicit conversion
x = "50"
y = int(x)  # String to integer
print("Result of explicit conversion:", y, "| Type:", type(y))
```

Result:

```
Result of implicit conversion: 7.5 | Type: <class 'float'>
Result of explicit conversion: 50 | Type: <class 'int'>

=== Code Execution Successful ===
```

3. Conditional Statements

3.1 Check if a number is positive, negative, or zero

Code:

```
number = int(input("Enter a number: "))
if number > 0:
    print("The number is positive.")
elif number < 0:
    print("The number is negative.")
else:
    print("The number is zero.")
```

Result:

```
Enter a number: 27
The number is positive.

=== Code Execution Successful ===
```

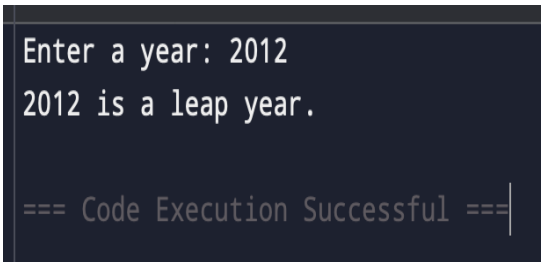
3.2 Determine if a given year is a leap year

Code:

```
year = int(input("Enter a year: "))
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print(year, "is a leap year.")
else:
```

```
    print(year, "is not a leap year.")
```

Result:

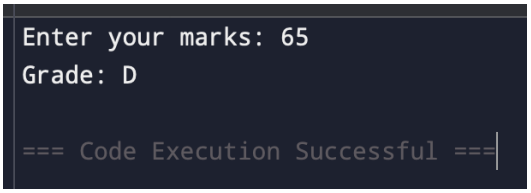


3.3 Simple grade calculator based on marks

Code:

```
marks = int(input("Enter your marks: "))
if marks >= 90:
    print("Grade: A")
elif marks >= 80:
    print("Grade: B")
elif marks >= 70:
    print("Grade: C")
elif marks >= 60:
    print("Grade: D")
else:
    print("Grade: F")
```

Result:



4. Loops

4.1 Print numbers from 1 to 100

Code:

```
for i in range(1, 101):
    print(i, end=" ")
```

Result:

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28
    29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52
    53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76
    77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98 99
    100
=== Code Execution Successful ===
```

4.2 Multiplication table generator

Code:

```
num = int(input("Enter a number: "))
for i in range(1, 11):
    print(f"{num} x {i} = {num * i}")
```

Result:

```
Enter a number: 8
8 x 1 = 8
8 x 2 = 16
8 x 3 = 24
8 x 4 = 32
8 x 5 = 40
8 x 6 = 48
8 x 7 = 56
8 x 8 = 64
8 x 9 = 72
8 x 10 = 80

=== Code Execution Successful ===
```

4.3 Sum of all even numbers up to a given limit

Code:

```
limit = int(input("Enter the limit: "))
```

```
even_sum = sum(i for i in range(2, limit + 1, 2))
print("Sum of even numbers:", even_sum)
```

Result:

```
Enter the limit: 8
Sum of even numbers: 20

=== Code Execution Successful ===
```

5. Functions

5.1 Calculate the factorial of a number

Code:

```
def factorial(n):
    return 1 if n == 0 else n * factorial(n - 1)
num = int(input("Enter a number: "))
print("Factorial:", factorial(num))
```

Result:

```
Enter a number: 8
Factorial: 40320

=== Code Execution Successful ===
```

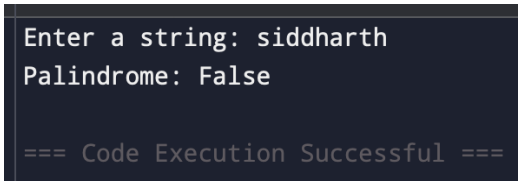
5.2 Check if a string is a palindrome

Code:

```
def is_palindrome(s):
    return s == s[::-1]

string = input("Enter a string: ")
```

```
print("Palindrome:", is_palindrome(string))
```

Result:

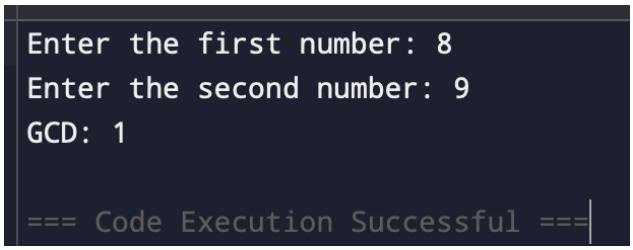


5.3 Find the greatest common divisor (GCD) of two numbers

Code:

```
from math import gcd
a = int(input("Enter the first number: "))
b = int(input("Enter the second number: "))
print("GCD:", gcd(a, b))
```

Result:



6. Lists

6.1 Find the largest and smallest elements in a list

Code:

```
# Input: List of numbers
numbers = [12, 45, 2, 98, 23, 7]
# Find the largest and smallest elements
largest = max(numbers)
smallest = min(numbers)
# Display the results
```

```
print(f"The largest element is: {largest}")
print(f"The smallest element is: {smallest}")
```

Result:

```
The largest element is: 98
The smallest element is: 2

=== Code Execution Successful ===
```

6.2 Reverse a list without using built-in functions

Code:

```
numbers = [12, 45, 7, 22, 89]
reversed_list = []
for i in range(len(numbers) - 1, -1, -1):
    reversed_list.append(numbers[i])
print("Reversed list:", reversed_list)
```

Result:

```
Reversed list: [89, 22, 7, 45, 12]

=== Code Execution Successful ===
```

6.3 Remove duplicates from a list

Code:

```
numbers = [12, 45, 7, 12, 89, 45]
unique_numbers = list(set(numbers))
print("Unique numbers:", unique_numbers)
```

Result:

```
Unique numbers: [89, 12, 45, 7]

=== Code Execution Successful ===
```

7. Strings

7.1 Count the number of vowels in a string

Code:

```
string = input("Enter a string: ")
vowels = "aeiouAEIOU"
count = sum(1 for char in string if char in vowels)
print("Number of vowels:", count)
```

Result:

```
Enter a string: SIDDHARTH
Number of vowels: 2

=== Code Execution Successful ===
```

7.2 Reverse a string without slicing

Code:

```
string = input("Enter a string: ")
reversed_string = ""
for char in string:
    reversed_string = char + reversed_string
print("Reversed string:", reversed_string)
```

Result:

```
Enter a string: siddharth
Reversed string: htrahddis

=== Code Execution Successful ===
```

7.3 Frequency of each character in a string

Code:

```
from collections import Counter
string = input("Enter a string: ")
frequency = Counter(string)
print("Character frequency:", dict(frequency))
```

Result:

```
Enter a string: siddharth
Character frequency: {'s': 1, 'i': 1, 'd': 2, 'h': 2, 'a': 1, 'r': 1, 't':
    1}

=== Code Execution Successful ===
```

8. Dictionaries

8.1 Store student names and grades, and display highest and lowest grades

Code:

```
students = {"Siddharth": 85, "Kanchan": 92, "Mehar": 78}
print("Highest grade:", max(students.values()))
print("Lowest grade:", min(students.values()))
```

Result:

```
Highest grade: 92
Lowest grade: 78

=== Code Execution Successful ===
```

8.2 Count occurrences of each word in a sentence

Code:

```
from collections import Counter
# Input sentence
sentence = input("Enter a sentence: ")
# Split the sentence into words
words = sentence.split()
# Count occurrences of each word
word_counts = Counter(words)
# Display the result
print("Word occurrences:")
for word, count in word_counts.items():
    print(f"{word}: {count}")
```

Result:

```
Enter a sentence: HARD HARD
Word occurrences:
HARD: 2

=== Code Execution Successful ===
```

8.3 Merge two dictionaries and sort by keys

Code:

```
dict1 = {"a": 10, "c": 30}
dict2 = {"b": 20, "d": 40}
merged = {**dict1, **dict2}
sorted_dict = dict(sorted(merged.items()))
print("Sorted dictionary:", sorted_dict)
```

Result:

```
Sorted dictionary: {'a': 10, 'b': 20, 'c': 30, 'd': 40}

=== Code Execution Successful ===
```

9. File Handling

9.1 Write data to a file and read it back

Code:

Result:

9.2 Count lines, words, and characters in a file

9.3 Append new data to an existing file

10. Exception Handling

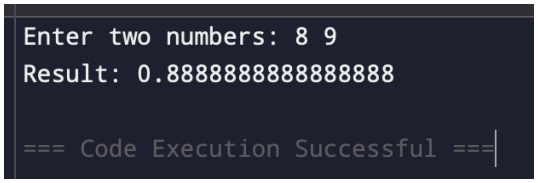
10.1 Handle division by zero

Code:

```
try:
    a, b = map(int, input("Enter two numbers: ").split())
    result = a / b
```

```
    print("Result:", result)
except ZeroDivisionError:
    print("Error: Division by zero is not allowed.")
```

Result:

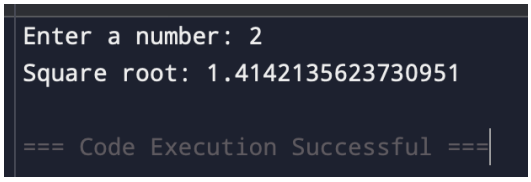


10.2 Handle invalid inputs for square roots

Code:

```
import math
try:
    num = int(input("Enter a number: "))
    if num < 0:
        raise ValueError("Square root of negative numbers is undefined.")
    print("Square root:", math.sqrt(num))
except ValueError as e:
    print("Error:", e)
```

Result:



10.3 Custom exception for age verification in a voting system

Code:

```
class AgeException(Exception):
    pass
age = int(input("Enter your age: "))
try:
    if age < 18:
```

```
        raise AgeException("You must be 18 or older to vote.")
    print("You are eligible to vote.")
except AgeException as e:
    print("Error:", e)
```

Result:

```
Enter your age: 21
You are eligible to vote.

=== Code Execution Successful ===
```

11. Classes and Objects

11.1 Class for a rectangle

Code:

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width
    def area(self):
        return self.length * self.width
    def perimeter(self):
        return 2 * (self.length + self.width)
rect = Rectangle(10, 5)
print("Area:", rect.area())
print("Perimeter:", rect.perimeter())
```

Result:

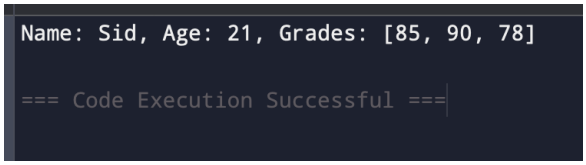
```
Area: 50
Perimeter: 30

=== Code Execution Successful ===
```

11.2 Student class

Code:

```
class Student:
    def __init__(self, name, age, grades):
        self.name = name
        self.age = age
        self.grades = grades
    def display_details(self):
        print(f"Name: {self.name}, Age: {self.age}, Grades: {self.grades}")
student = Student("Sid", 21, [85, 90, 78])
student.display_details()
```

Result:

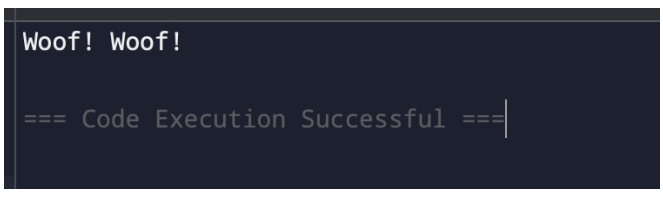


11.3 Inheritance example

Code:

```
class Animal:
    def sound(self):
        print("This is a generic animal sound.")
class Dog(Animal):
    def sound(self):
        print("Woof! Woof!")
dog = Dog()
dog.sound()
```

Result:



12. ReLU Function

Code:

```
def relu(x):
    return x if x >= 0 else 0
num = float(input("Enter a number: "))
print("ReLU result:", relu(num))
```

Result: